

**PERANCANGAN APLIKASI PENGELOLAAN DATA KUCING
BERBASIS WEB MENGGUNAKAN LARAVEL DAN REACT**



Disusun oleh :

Nama : Serlia Wati

NIM : 241054001

**PROGAM STUDI INFORMATIKA
FAKULTAS SAINS DAN TEKNOLOGI INFORMASI
UNIVERSITAS AKPRIND INDONESIA**

2026/2027

A. Teknologi yang dipilih (frontend & backend) beserta alasannya

a.) Backend: Laravel (PHP)

Pada sisi backend, aplikasi menggunakan Laravel sebagai framework PHP untuk membangun REST API. Laravel dipilih karena memiliki struktur kode yang rapi dengan konsep Model-View-Controller (MVC) sehingga memudahkan proses pengembangan dan pemeliharaan aplikasi. Selain itu, Laravel menyediakan fitur Eloquent ORM yang mempermudah pengelolaan database, proses validasi data, routing, serta menghasilkan response dalam format JSON yang sesuai untuk kebutuhan REST API.

b.) Frontend: React.js

Pada sisi frontend, aplikasi menggunakan React.js untuk membangun antarmuka pengguna (User Interface). React dipilih karena mampu menghasilkan tampilan yang interaktif dan responsif melalui konsep component-based, sehingga setiap komponen dapat digunakan kembali dan lebih mudah dikelola. React juga memudahkan komunikasi dengan backend melalui REST API sehingga data dapat ditampilkan secara dinamis tanpa perlu melakukan refresh halaman.

c.) Styling: Tailwind CSS

Untuk desain antarmuka, aplikasi menggunakan Tailwind CSS. Framework ini dipilih karena menyediakan berbagai utility class yang mempermudah proses pembuatan tampilan yang modern, responsif, dan konsisten. Dengan Tailwind CSS, proses pengembangan antarmuka menjadi lebih cepat tanpa perlu menulis banyak kode CSS secara manual.

d.) Database: MySQL

Aplikasi menggunakan MySQL sebagai sistem manajemen basis data. MySQL dipilih karena mudah diintegrasikan dengan Laravel, mampu menyimpan data secara terstruktur, serta mendukung operasi CRUD dengan performa yang baik sehingga sesuai untuk kebutuhan aplikasi pengelolaan data kucing.

e.) HTTP Client: Axios

Untuk menghubungkan frontend dengan backend, aplikasi menggunakan Axios sebagai HTTP Client. Axios digunakan untuk mengirim dan menerima data melalui REST API menggunakan metode HTTP seperti GET, POST, PUT, dan DELETE, sehingga proses pertukaran data antara React dan Laravel dapat dilakukan dengan mudah dan efisien.

f) Web Server: XAMPP

Selama proses pengembangan aplikasi, digunakan XAMPP sebagai web server lokal. XAMPP dipilih karena telah menyediakan Apache, PHP, dan MySQL dalam satu paket sehingga memudahkan proses menjalankan aplikasi Laravel serta pengelolaan database pada lingkungan pengembangan lokal.

B. Penjelasan setiap endpoint API (method, URL, request, response)

a) Endpoint Menampilkan Seluruh Data Kucing

Endpoint ini digunakan untuk mengambil seluruh data kucing yang tersimpan pada database. Endpoint ini juga mendukung fitur search dan pagination menggunakan query parameter.

- **Method** -> GET
- **URL** -> <http://127.0.0.1:8000/api/cats>
- **Query Parameter** -> page, limit, search
- **Request Header**

Docs Params Authorization Headers 10 Body Scripts T

Headers 8 hidden

Key	Value
  Accept	application/json
 Content-Type	application/json
Key	Value

- **Request Body** -> Tidak ada
- **Request**

```
GET /api/cats?page=1&limit=5&search=Persia
Accept: application/json
```

- **Response**

```
{
  "success": true,
  "message": "Data kucing berhasil diambil",
  "data": {
    "current_page": 1,
    "data": [
      {
        "id": 1,
        "nama": "Milo",
        "ras": "Persia",
        "warna": "Putih",
        "umur": 2,
        "jenis_kelamin": "Jantan",
        "berat": "3.5",
        "status_vaksin": "Sudah"
      }
    ]
  }
}
```

b). Endpoint Menambahkan Data Kucing

Endpoint ini digunakan untuk menambahkan data kucing baru ke dalam database.

- Method -> POST
- URL -> <http://127.0.0.1:8000/api/cats>
- Request Header

≡ Docs Params Authorization Headers 10 Body ● Scripts ● T

Headers 8 hidden

Key	Value
✓ Accept	application/json
✓ Content-Type	application/json
Key	Value

- Request Body

```
{
  "nama": "Mochi",
  "ras": "Anggora",
  "warna": "Putih",
  "umur": 1,
  "jenis_kelamin": "Betina",
  "berat": 2.8,
  "status_vaksin": "Sudah"
}
```

- Response

```
{
  "success": true,
  "message": "Data kucing berhasil ditambahkan",
  "data": {
    "id": 6,
    "nama": "Mochi",
    "ras": "Anggora",
    "warna": "Putih",
    "umur": 1,
    "jenis_kelamin": "Betina",
    "berat": 2.8,
    "status_vaksin": "Sudah"
  }
}
```

c). Endpoint Mengubah Data Kucing

Endpoint ini digunakan untuk memperbarui data kucing berdasarkan ID.

- Method -> PUT
- URL -> <http://127.0.0.1:8000/api/cats/{id}>
- Request Header

≡ Docs Params Authorization Headers 10 Body ● Scripts ● T

Headers 8 hidden

Key	Value
✓ Accept	application/json
✓ Content-Type	application/json
Key	Value

- Request Body

```
{
  "nama": "Mochi",
  "ras": "Anggora",
  "warna": "Cream",
  "umur": 2,
  "jenis_kelamin": "Betina",
  "berat": 3.1,
  "status_vaksin": "Sudah"
}
```

- Response

```
{
  "success": true,
  "message": "Data kucing berhasil diupdate",
  "data": {
    "id": 6,
    "nama": "Mochi",
    "ras": "Anggora",
    "warna": "Cream",
    "umur": 2,
    "jenis_kelamin": "Betina",
    "berat": 3.1,
    "status_vaksin": "Sudah"
  }
}
```

d) Endpoint Menghapus Data Kucing

Endpoint ini digunakan untuk menghapus data kucing berdasarkan ID.

- Method -> DELETE
- URL -> <http://127.0.0.1:8000/api/cats/{id}>
- Request Header

≡ Docs Params Authorization Headers 10 Body ● Scripts ● T

Headers 8 hidden

Key	Value
✓ Accept	application/json
✓ Content-Type	application/json
Key	Value

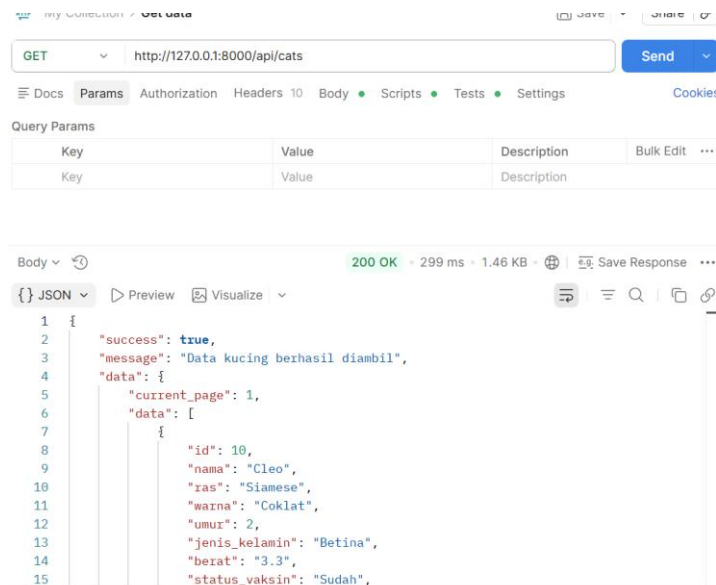
- Request Body -> Tidak ada
- Response

```
{
  "success": true,
  "message": "Data kucing berhasil dihapus"
}
```

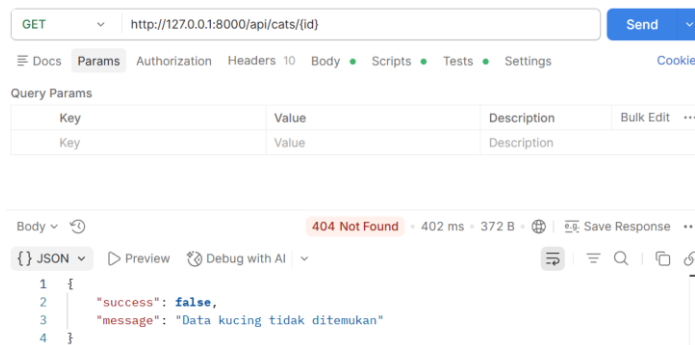
C. Screenshot hasil testing setiap endpoint di Postman

a) Method GET

- Testing Sukses

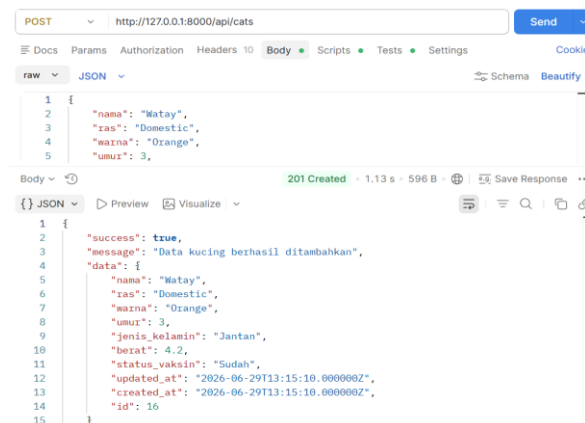


- Testing Gagal

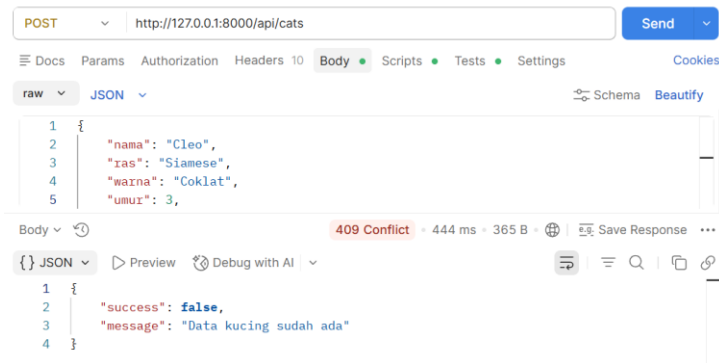


b) Method POST

- Testing Sukses

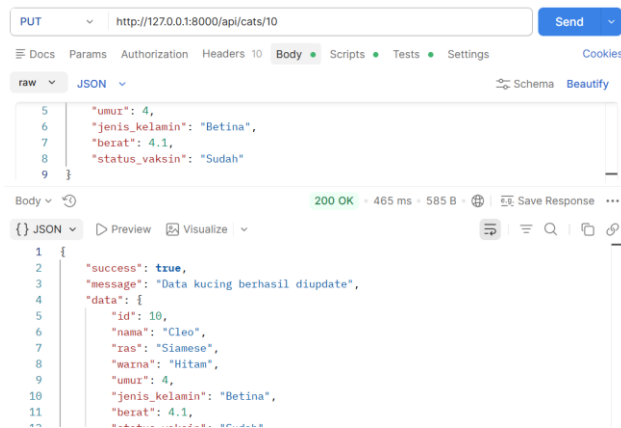


- **Testing Gagal**

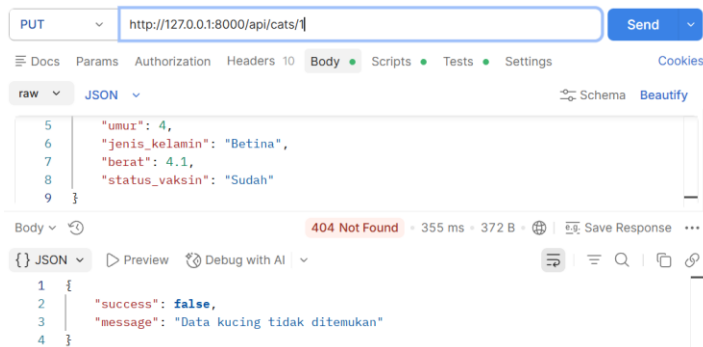


c) Method PUT

- **Testing Sukses**

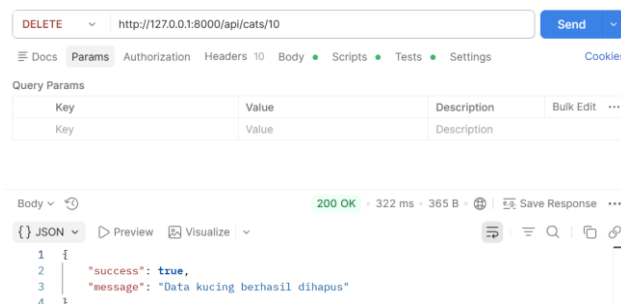


- **Testing Gagal**

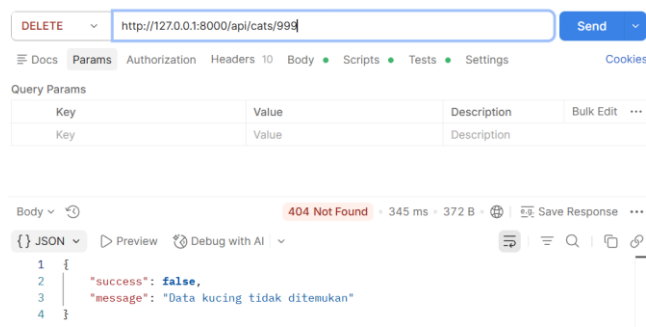


d) Method DELETE

- **Testing Sukses**

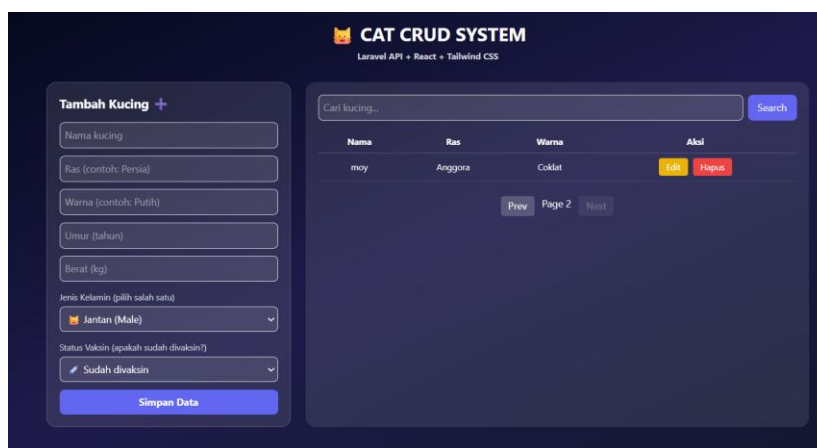
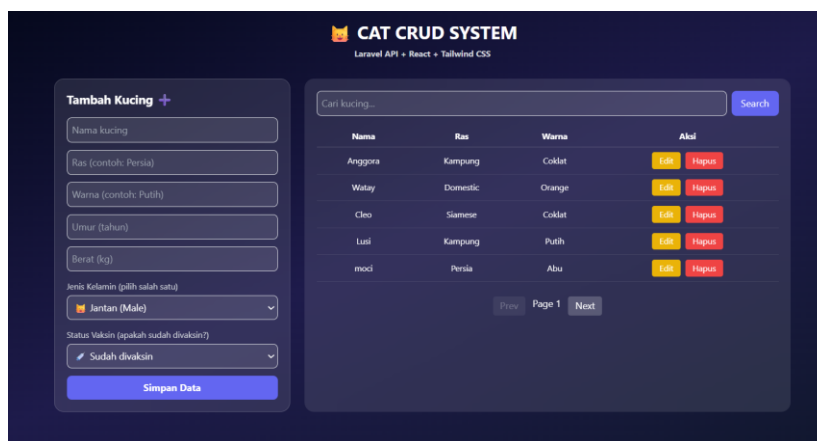


- **Testing Gagal**

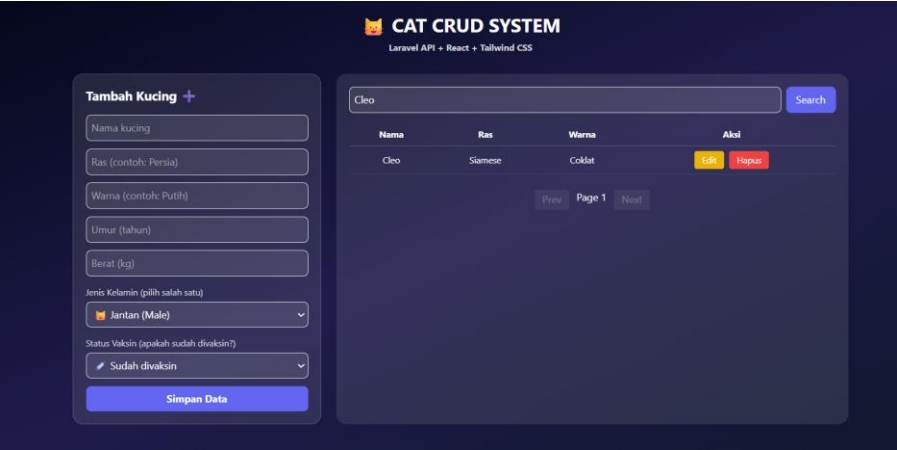


D. Screenshot tampilan UI frontend (halaman DataTable, form tambah, form edit, dll.)

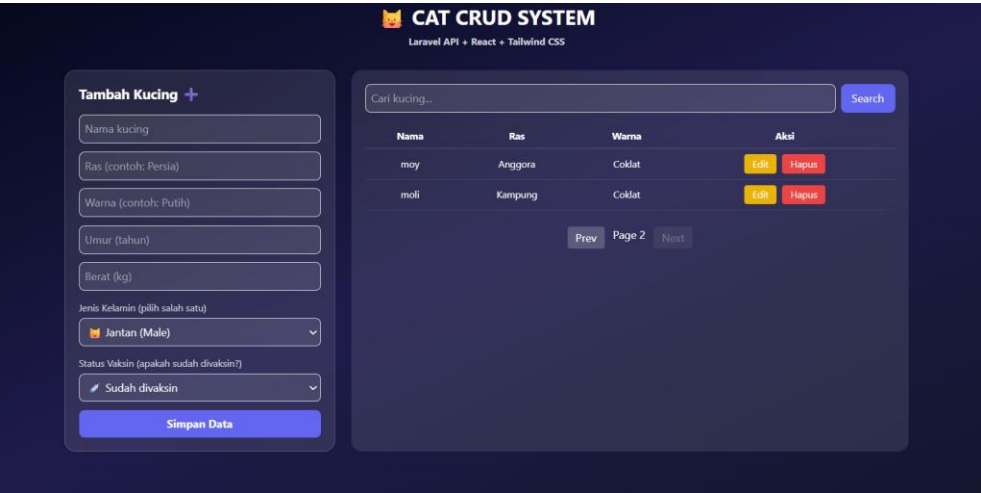
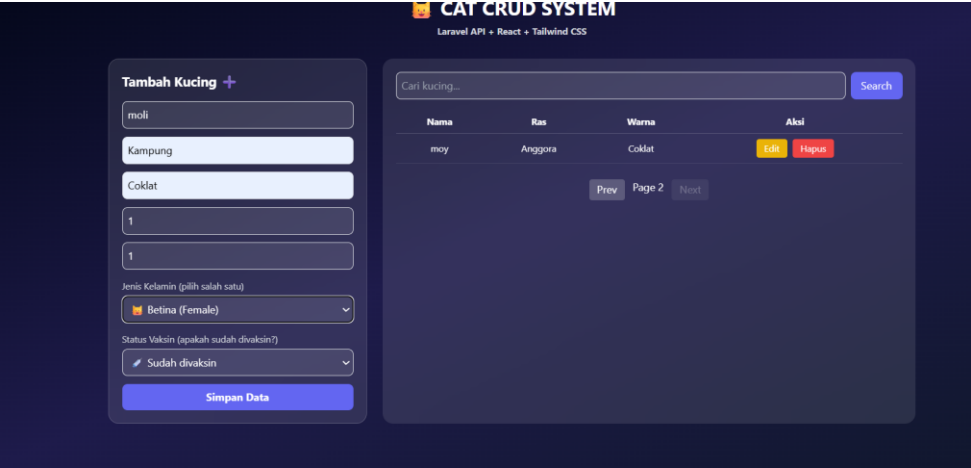
Halaman Data Table



Coba Search nama kucing CLEO:



Testing CRUD :
Tambah Data



Edit Data :

The screenshot displays the 'CAT CRUD SYSTEM' interface. On the left, a modal titled 'Edit Data' is open, showing a form with the following fields: 'chi' (text), 'Domestic' (text), 'Orange' (text), '3' (text), and '4,2' (text). Below these fields, there is a dropdown menu for 'Jenis Kelamin (pilih salah satu)' with 'Jantan (Male)' selected. Below the dropdown is a status message: 'Status Vaksin (apakah sudah divaksin?)' with a dropdown menu showing 'Sudah divaksin'. At the bottom of the modal are two buttons: 'Update Data' (blue) and 'Cancel' (gray). On the right, the main data table is visible. It has a search bar at the top with the placeholder 'Cari kucing...'. The table has five columns: 'Nama', 'Ras', 'Warna', and 'Aksi'. The 'Aksi' column contains 'Edit' (yellow) and 'Hapus' (red) buttons. The table contains five rows of data: 1. Anggora, Kampung, Coklat, Edit, Hapus; 2. Watay, Domestic, Orange, Edit, Hapus; 3. Cleo, Siamese, Coklat, Edit, Hapus; 4. Luni, Kampung, Putih, Edit, Hapus; 5. mod, Persia, Abu, Edit, Hapus. Below the table, there are navigation buttons: 'Prev', 'Page 1', and 'Next'.

Hapus Data

CAT CRUD SYSTEM
Laravel API + React + Tailwind CSS

Tambah Kucing +

Nama kucing

Ras (contoh: Persia)

Warna (contoh: Putih)

Umur (tahun)

Berat (kg)

Jenis Kelamin (pilih salah satu)

Jantan (Male)

Status Vaksin (apakah sudah divaksin?)

Sudah divaksin

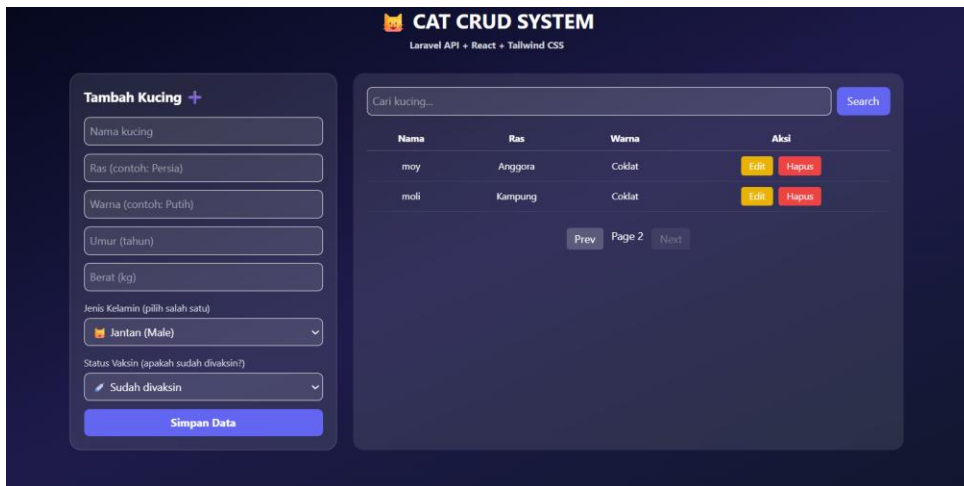
Simpan Data

Cari kucing...

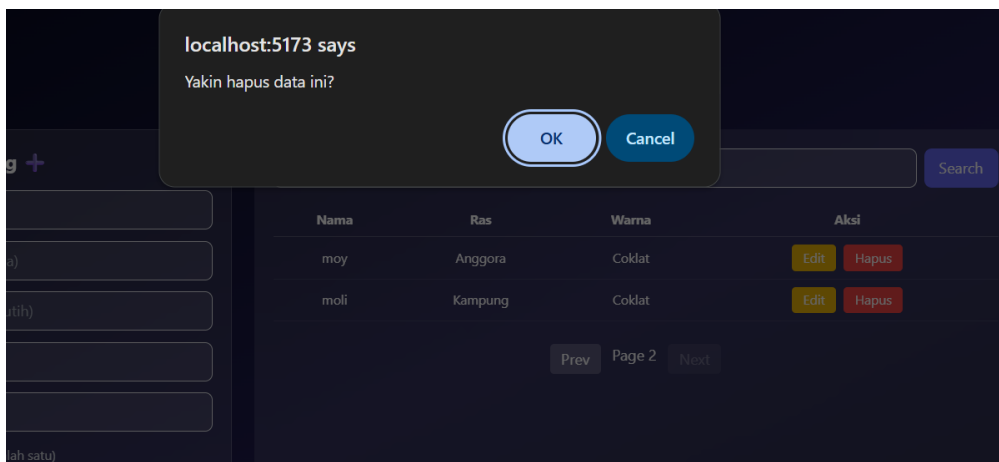
Search

Nama	Ras	Warna	Aksi	
Anggora	Kampung	Coklat	EDR	Hapus
chi	Domestic	Orange	EDR	Hapus
Cleo	Siamese	Coklat	EDR	Hapus
Lusi	Kampung	Putih	EDR	Hapus
moci	Persia	Abu	EDR	Hapus

Prev Page 1 Next



Hapus Data MOLI:



Data MOLI Hilang/Hapus :

